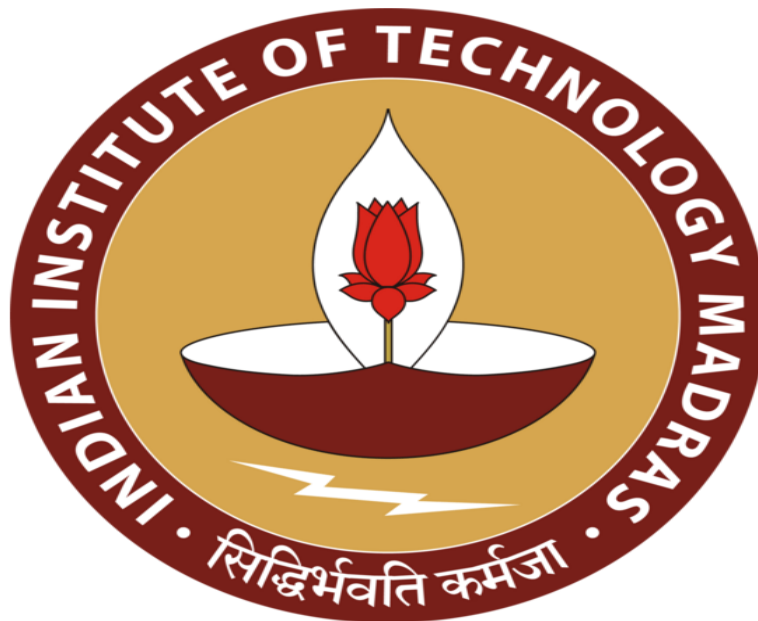


SOFTWARE ENGINEERING PROJECT MAY-2026
MILESTONE 3 DELIVERABLES FOR COMMUNITY SERVICES PLATFORM
SUBDOMAIN: APARTMENT ASSOCIATION

Submitted By

Team Name: Sputnik
Team Code: MAY2026-Team-095



IITM Online BS Degree Program,
Indian Institute of Technology, Madras, Chennai

Tamil Nadu, India, 600036

SMART SOCIETY COMPLAINT RESOLUTION SYSTEM

Project Details

Chosen Problem Statement: Community Services Platform

Subdomain: Apartment Association

Software Name: Smart Society Complaint Resolution System

Date of Submission: 02-08-2026

Team Details

Team Name: Sputnik

Team Code: MAY2026-Team-095

Name	Roll Number	Role
P Baskaran Nadar	24f2008200	Project Manager
Rahul Khanna GK	24f1002269	Backend Developer
Jishan Kapoor	24f1002364	Frontend Developer
Annapoorani Muthuveerappan	23f1001421	Scrum Master
Roshan Ashwin Singh	22f1000684	Quality Assurance

API ENDPOINTS, TEST CASES AND USER TESTING

List of user stories selected for Milestone 3 and the created API'S.

From the 17 user stories identified in Milestone 1, 10 user stories were selected for implementation in Milestone 3. The corresponding APIs developed to fulfill each of these selected user stories are presented in the table below.

SL NO	USER STORY	List of API's created
1	As a Resident, I want to submit a complaint with its description, category, and photos, so that maintenance issues can be reported accurately and quickly.	POST/COMPLAINTS GET/CATEGORIES
2	As a Resident, I want to receive a unique complaint reference number after submitting a complaint, so that I can track its progress.	POST/COMPLAINTS
3	As a Society Manager, I want complaints to be categorized automatically (Electrical, Plumbing, Security, etc.), so that issues can be assigned to the correct personnel.	GET/CATEGORIES POST/ADMIN/CATEGORIES
4	As a Resident, I want to view the real time status of my complaint, so that I know the progress of issue resolution.	GET/COMPLAINTS/{COMPLAINT_ID}
5	As a Resident, I want to receive notifications whenever my complaint status changes, so that I stay informed without contacting management.	GET/NOTIFICATIONS PUT/NOTIFICATIONS/{NOTIFICATION-ID}/READ PUT/NOTIFICATIONS/READ-ALL
6	As a Society Manager, I want to assign complaints to maintenance staff based on their expertise, so that issues are resolved by qualified personnel.	POST/ADMIN/STAFF PUT/ADMIN/COMPLAINTS/{COMPLAINT-ID}
7	As a Maintenance staff, I want to receive digital work orders with complaint details, priority, resident information and location so that I clearly understand the maintenance requirements.	GET/COMPLAINTS/{COMPLAINT-ID}
8	As a Maintenance staff, I want to update the status of assigned tasks, so that management and residents can track progress effectively.	PUT/STAFF/COMPLAINTS/{COMPLAINT-ID}/STATUS
9	As a Resident, I want to rate the quality of maintenance work, so that management can evaluate service performance.	POST/COMPLAINTS/{COMPLAINT-ID}/FEEDBACK

14	As a Chairman, I want to view a dashboard showing pending, resolved, and overdue complaints, so that I can monitor the overall maintenance performance of the association.	GET/ADMIN/DASHBOARD
----	--	---------------------

API ENDPOINT DESCRIPTION

Each API endpoint implemented for the selected user stories is documented in a structured format.

The documentation includes the API endpoint, the corresponding user story, the URL being tested, a brief description of the API, the test case(s) with their inputs, expected and actual outputs, the test results, and a screenshot of the API execution in Swagger.

In addition to the individual API testing performed through Swagger, a comprehensive automated API validation was carried out using the pytest -v framework.

The screenshots of the complete pytest execution, along with the generated HTML test report (pytest_report.html), have been included at the end of this document to demonstrate the successful execution and validation of all implemented APIs.

This provides a comprehensive record of the API implementation and testing performed as part of Milestone 3.

API ENDPOINT : POST/auth/register

USER STORY: General

URL BEING TESTED: http://127.0.0.1:5000/auth/register

DESCRIPTION: This API is used to register a new **Resident** in the Smart Society Management System. It accepts the resident's personal information, apartment details, and password, validates the input, creates a new resident account, and returns a JWT access token along with the registered user's profile information upon successful registration.

TEST CASE 1:

INPUTS: {
 "name": "Rakesh Kumar",
 "email": "rakesh@example.com",
 "mobile_number": "9004001235",
 "password": "Password@123",
 "flat_number": "A402",
 "building": "Block A"
 }

EXPECTED OUTPUT:

- HTTP Status Code: **201 Created**
- Resident account should be created successfully.

- A unique Resident ID should be generated.
- JWT Access Token should be returned.
- Registered resident details should be returned.
- Success message: **"Registration successful."**

ACTUAL OUTPUT:

- HTTP Status Code: 201 Created
- Resident account created successfully.
- Resident ID 17 generated.
- JWT Access Token returned.
- Resident details returned:
 - Name: Rakesh Kumar
 - Email: rakesh@example.com
 - Mobile Number: 9004001235
 - Role: RESIDENT
 - Flat Number: A402
 - Building: Block A
 - Account Status: Active
- Success message: "Registration successful."

RESULT: SUCCESS

TEST CASE 2:

INPUTS: {

```
"name": "Rakesh Kumar",
"email": "rakesh@example.com",
"mobile_number": "9004001235",
"password": "Password123",
"flat_number": "A402",
"building": "Block A"
}
```

EXPECTED OUTPUT:

- HTTP Status Code: 400 Bad Request
- Registration should fail because the password does not satisfy the password policy.
- The API should return a validation error indicating that the password must contain at least one special character.
- No resident account should be created.

ACTUAL OUTPUT

- HTTP Status Code: 400 Bad Request

- Registration request rejected.
- Validation error returned
 - Password must contain at least one special character.
- No resident account was created.

RESULT: FAIL

TEST CASE 3:

INPUTS: {
 "name": "Rakesh Kumar",
 "email": "rakeshexample.com",
 "mobile_number": "9004001235",
 "password": "Password@123",
 "flat_number": "A402",
 "building": "Block A"
 }

EXPECTED OUTPUT:

- HTTP Status Code: 400 Bad Request
- Registration should fail because the email address is not in a valid format.
- The API should return a validation error indicating that the email address is invalid.
- No resident account should be created.

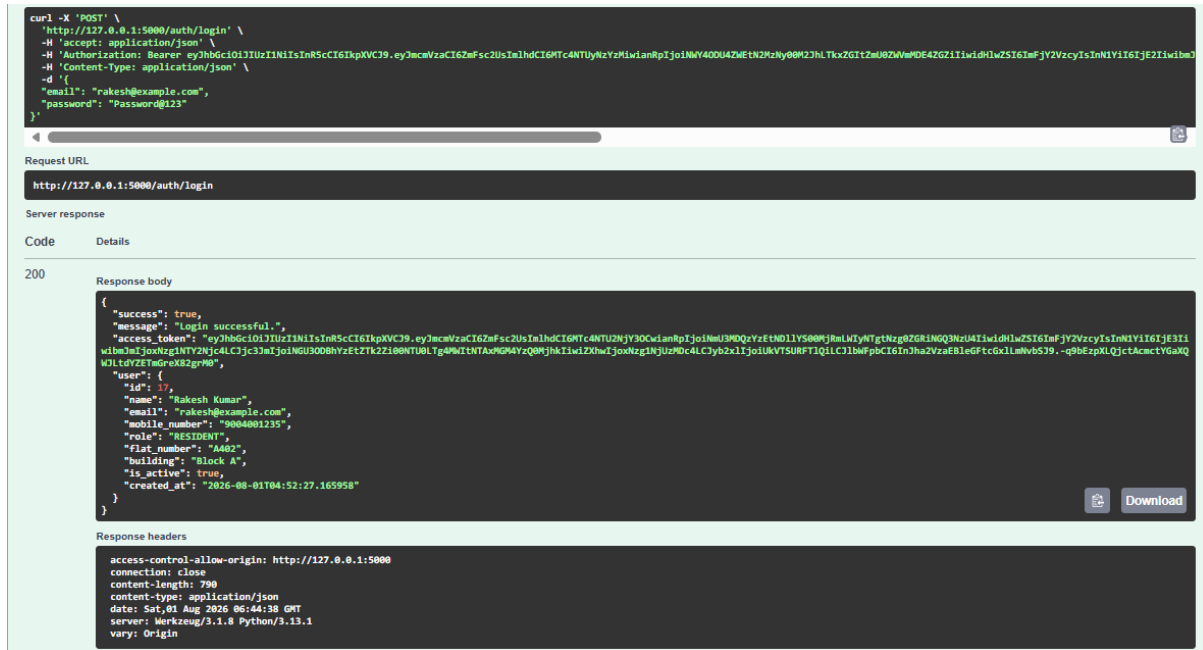
ACTUAL OUTPUT

- HTTP Status Code: 400 Bad Request
- Registration request rejected.
- Validation error returned
 - Not a valid email address.
- No resident account was created.

RESULT: FAIL

- Success message: "Login successful."

RESULT: SUCCESS



TEST CASE 2: (For Resident)

```
INPUTS: {
    "email": "rakesh@example.com",
    "password": "Passssword@123"
}
```

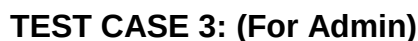
EXPECTED OUTPUT:

- HTTP Status Code: **401 Unauthorized**
- Login request should be rejected because the password is incorrect.
- The API should return an authentication error message.
- No JWT access token should be generated.
- User profile details should not be returned.

ACTUAL OUTPUT

- HTTP Status Code: **401 Unauthorized**
- Login request rejected.

- RESULT: FAIL**



EXPECTED OUTPUT:

- ## ACTUAL OUTPUT

- HTTP Status Code: 200 OK
- Administrator authenticated successfully.
- JWT access token generated.
- Administrator details returned:
 - User ID: 12

- Success message: **"Login successful."**

[illegible]

```
INPUTS: {
    "email": "staff2@smartsociety.com",
    "password": "Staff@123"
}
```

- HTTP Status Code: 200 OK
- Staff member should be authenticated successfully.
- JWT access token should be generated.
- Staff profile details should be returned.

- ## ACTUAL OUTPUT

- API ENDPOINT :** POST/auth/forgot-password

USER STORY: General

URL BEING TESTED: <http://127.0.0.1:5000/auth/register>

[illegible]

API ENDPOINT : GET/categories

USER STORY: User Story 3(As a Manager, I want complaints to be categorized automatically (Electrical, Plumbing, Security, etc.), so that issues can be assigned to the correct personnel.)

URL BEING TESTED: http://127.0.0.1:5000/categories

DESCRIPTION: This API retrieves all **active complaint categories** available in the Smart Society Management System. These categories help residents classify their complaints (e.g., Water Leakage, Electrical, Plumbing, Security, Parking, etc.) so that complaints can be routed to the appropriate maintenance staff. Only authenticated users can access this endpoint.

TEST CASE 1:

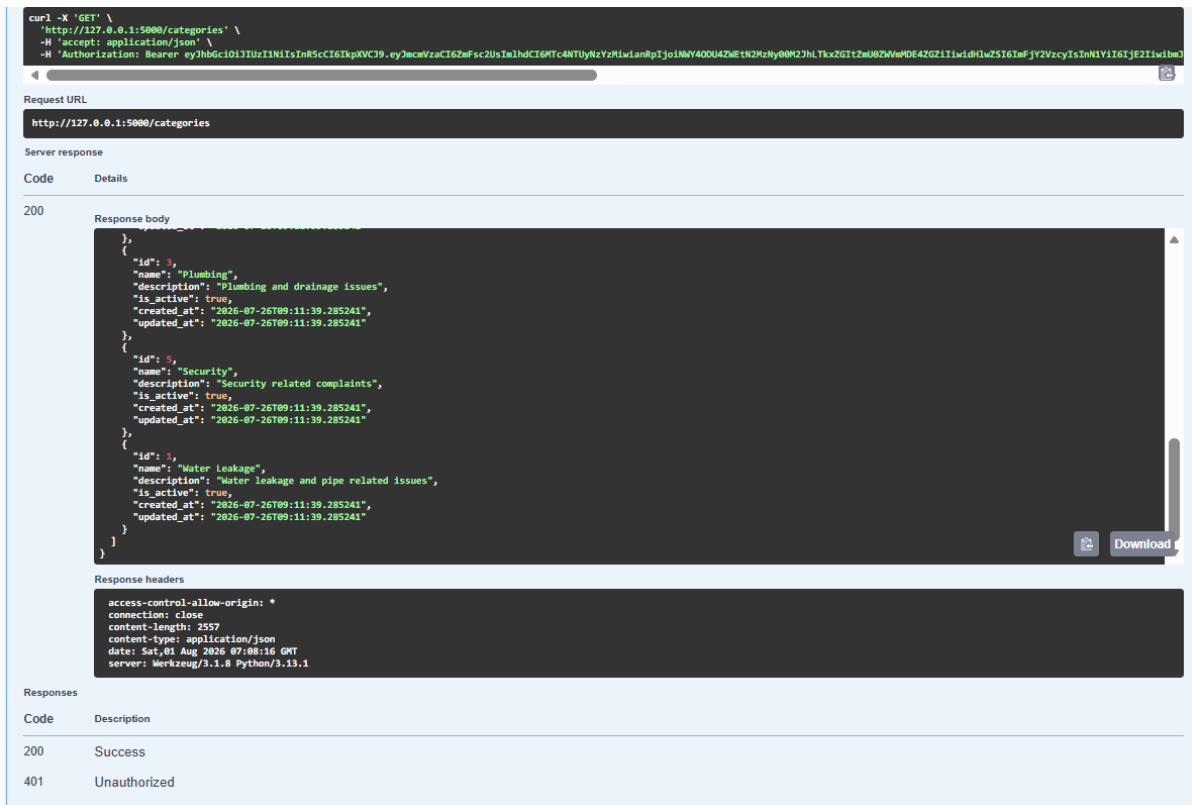
EXPECTED OUTPUT:

- HTTP Status Code: **200 OK**
- The API should return a list of all active complaint categories.
- Each category should include:
 - Category ID
 - Category Name
 - Description
 - Active Status
 - Created Date
 - Updated Date
- Response should indicate that the request was successful.

ACTUAL OUTPUT:

- HTTP Status Code: **200 OK**
- Request processed successfully.
- The API returned **9 active complaint categories**, including:
 - Water Leakage
 - Electrical
 - Plumbing
 - Cleanliness
 - Security
 - Parking
 - Lift/Elevator
 - Fire Safety
 - Gardening
- Each category included its **ID, name, description, active status, created_at, and updated_at** fields.
- Response contained "success": true.

RESULT: SUCCESS



API ENDPOINT : GET/auth/profile

USER STORY: General

URL BEING TESTED: http://127.0.0.1:5000/auth/profile

DESCRIPTION: This API retrieves the profile information of the currently authenticated user. It validates the JWT access token provided in the request header and returns the user's profile details, including personal information, role, apartment details, and account status. The endpoint can be accessed by any authenticated user (Resident, Staff, or Admin).

TEST CASE 1:

EXPECTED OUTPUT:

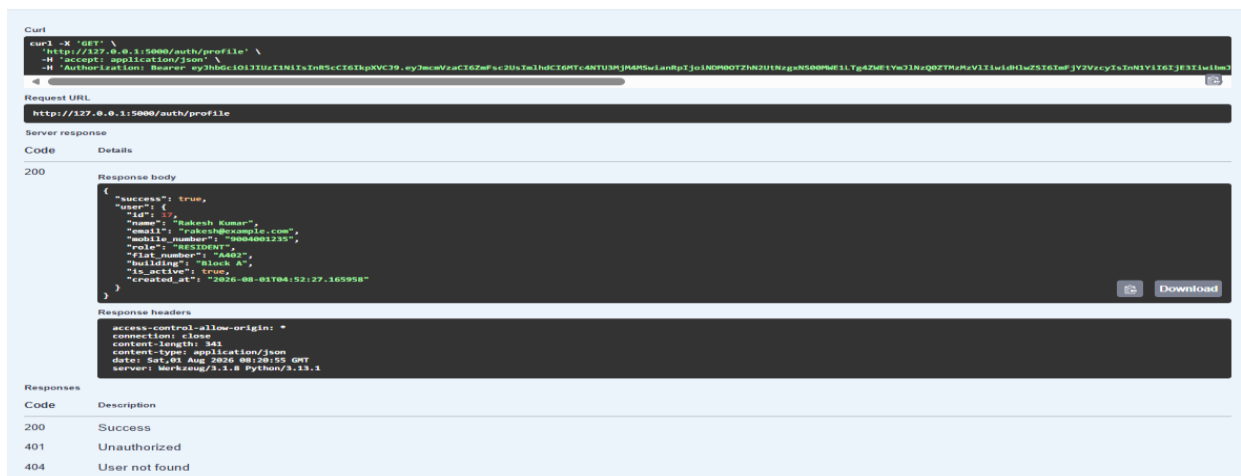
- HTTP Status Code: **200 OK**
- The authenticated user's profile information should be returned.
- The response should include:
 - User ID
 - Name
 - Email Address
 - Mobile Number
 - Role
 - Flat Number
 - Building Name
 - Account Status

- Account Creation Date
- Response should indicate successful retrieval of the profile.

ACTUAL OUTPUT:

- HTTP Status Code: **200 OK**
- Resident profile returned successfully.
- User details returned:
 - **User ID:** 17
 - **Name:** Rakesh Kumar
 - **Email:** rakesh@example.com
 - **Mobile Number:** 9004001235
 - **Role:** RESIDENT
 - **Flat Number:** A402
 - **Building:** Block A
 - **Account Status:** Active
 - **Created At:** 2026-08-01T04:52:27.165958
 - Response contained "success": true

RESULT: SUCCESS



API ENDPOINT : POST/complaints

USER STORY: User Story 1(As a Resident,I want to submit a complaint with its description, category, and photos, so that maintenance issues can be reported accurately and quickly.)

User Story 2(As a Resident, I want to receive a unique complaint reference number after submitting a complaint, so that I can track its progress.)

URL BEING TESTED: `http://127.0.0.1:5000/complaints`

DESCRIPTION: This API allows a resident to register a new maintenance complaint by providing the complaint category, title, description, location, and priority level. Upon successful submission, the system creates a unique complaint reference number, assigns the complaint an **OPEN** status, stores the complaint details in the

database, and returns the complete complaint information. This API can only be accessed by authenticated residents.

TEST CASE 1:

INPUTS: {
 "category_id": 1,
 "title": "Water leakage in kitchen",
 "description": "There is continuous water leakage under the
 kitchen sink.",
 "location": "Flat A402, Kitchen",
 "priority": "MEDIUM"
 }

EXPECTED OUTPUT:

- HTTP Status Code: **201 Created**
- Complaint should be created successfully.
- A unique complaint reference number should be generated.
- Complaint status should be set to **OPEN**.
- Complaint details should be returned, including:
 - Complaint ID
 - Complaint Code
 - Category Details
 - Resident Details
 - Priority
 - Location
 - Creation Timestamp

ACTUAL OUTPUT:

- HTTP Status Code: **201 Created**
- Complaint created successfully.
- Complaint ID **20** generated.
- Complaint Code **CMP-2026-000010** generated.
- Complaint status set to **OPEN**.
- Complaint details returned successfully:
 - **Title:** Water leakage in kitchen
 - **Description:** There is continuous water leakage under the kitchen sink.
 - **Location:** Flat A402, Kitchen
 - **Priority:** MEDIUM
 - **Category:** Water Leakage
 - **Resident:** Rakesh Kumar (ID: 17)
 - **Assigned Staff:** None
 - **Attachments:** None
 - **Feedback:** None

- Success message: **"Complaint created successfully."**

RESULT: SUCCESS

The screenshot shows a web browser interface with a dark theme. The top section displays the curl command used for the POST request. Below it, the 'Request URL' is shown as 'http://127.0.0.1:5000/complaints'. The 'Server response' section shows a status code of 201. The 'Response body' is a JSON object with the following structure:

```
{
  "success": true,
  "message": "Complaint created successfully.",
  "complaint": {
    "id": 20,
    "complaint_code": "CMP-2026-000018",
    "title": "Water leakage in kitchen",
    "description": "There is continuous water leakage under the kitchen sink.",
    "location": "Flat A402, Kitchen",
    "priority": "MEDIUM",
    "status": "OPEN",
    "resident_id": 17,
    "assigned_staff_id": null,
    "category_id": 1,
    "created_at": "2026-08-01T08:28:33.866556",
    "updated_at": "2026-08-01T08:28:33.866556",
    "resolved_at": null,
    "category": {
      "id": 1,
      "name": "Water Leakage",
      "description": "Water leakage and pipe related issues",
      "is_active": true,
      "created_at": "2026-07-26T09:11:39.285241",
      "updated_at": "2026-07-26T09:11:39.285241"
    },
    "resident": {
      "id": 17,

```

The 'Response headers' section shows the following headers:

```
access-control-allow-origin: http://127.0.0.1:5000
connection: close
content-length: 1231
content-type: application/json
date: Sat, 01 Aug 2026 08:28:38 GMT
server: Werkzeug/3.1.8 Python/3.13.1
vary: Origin
```

The 'Responses' table at the bottom shows a single entry with a status code of 201 and a description of 'Complaint created'.

Code	Description
201	Complaint created

API ENDPOINT : GET /complaints/{complaint_id}

USER STORY: User Story 4(As a Resident,I want to view the real time status of my complaint, so that I know the progress of issue resolution.)

URL BEING TESTED: <http://127.0.0.1:5000/complaints/20>

DESCRIPTION: This API retrieves the complete details of a specific complaint using its Complaint ID. It returns the complaint information, complaint reference number, category details, resident details, assigned staff (if any), attachments, feedback, and the complaint timeline. Only authenticated users with appropriate access permissions can view the complaint details.

TEST CASE 1:

INPUTS:

- Input Parameter: Complaint ID-20

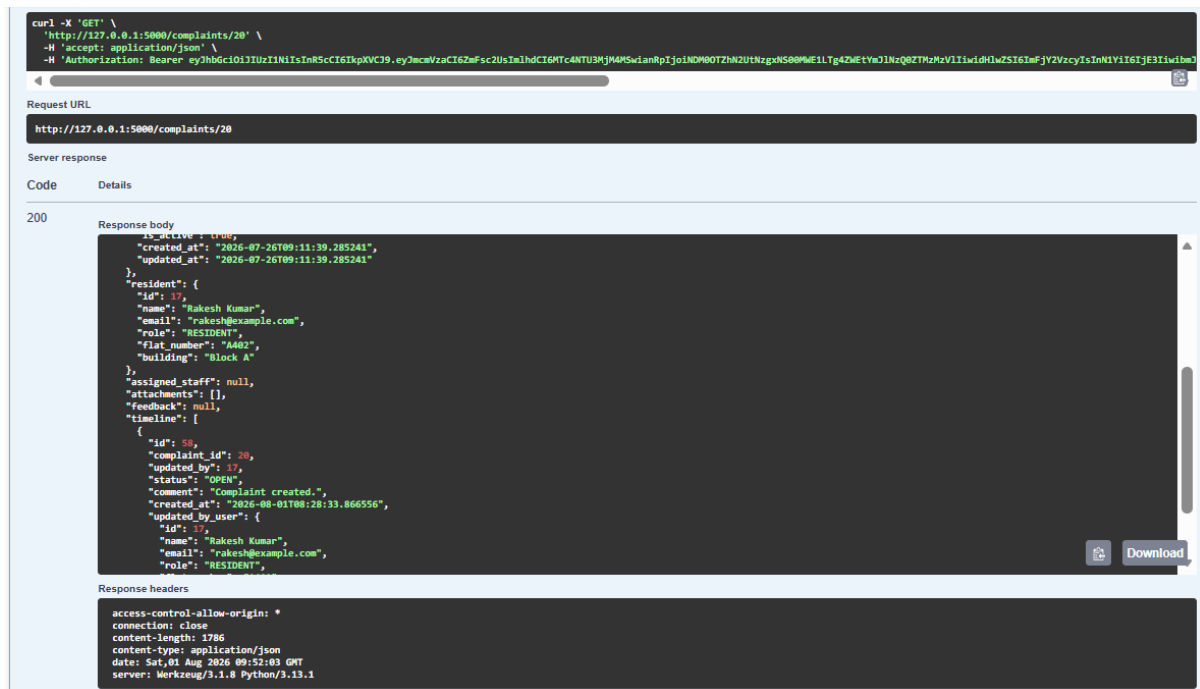
EXPECTED OUTPUT:

- HTTP Status Code: **200 OK**
- The complaint corresponding to Complaint ID 20 should be retrieved successfully.
- The response should include:
 - Complaint ID
 - Complaint Reference Number
 - Title
 - Description
 - Location
 - Priority
 - Status
 - Category Details
 - Resident Details
 - Assigned Staff Details (if assigned)
 - Attachments
 - Feedback
 - Complaint Timeline

ACTUAL OUTPUT:

- HTTP Status Code: 200 OK
- Complaint retrieved successfully.
- Complaint details returned:
 - Complaint ID: 20
 - Complaint Code: CMP-2026-000010
 - Title: Water leakage in kitchen
 - Description: There is continuous water leakage under the kitchen sink.
 - Location: Flat A402, Kitchen
 - Priority: MEDIUM
 - Status: OPEN
 - Category: Water Leakage
 - Resident: Rakesh Kumar (ID: 17)
 - Assigned Staff: None
 - Attachments: None
 - Feedback: None
 - Timeline: Initial complaint creation entry recorded with status OPEN and comment "Complaint created."
- Response contained "success": true

RESULT: SUCCESS



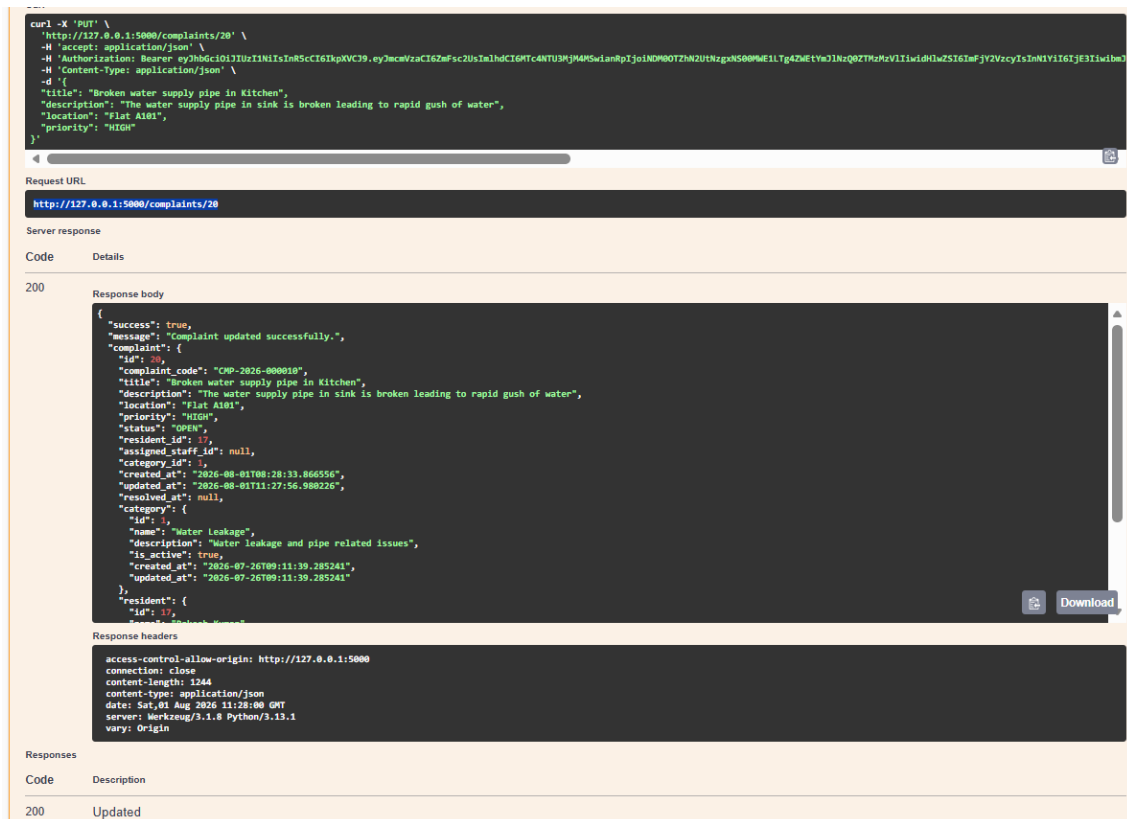
- HTTP Status Code: 200 OK
- Complaint should be updated successfully.
- The modified complaint details should be saved in the database.
- The response should return the updated complaint information, including:
 - Updated Title
 - Updated Description
 - Updated Location
 - Updated Priority

- Updated Timestamp
- Success message: **"Complaint updated successfully."**

ACTUAL OUTPUT:

- HTTP Status Code: 200 OK
- Complaint retrieved successfully.
- Complaint details returned:
 - Complaint ID: 20
 - Complaint Code: CMP-2026-000010
 - Title: Water leakage in kitchen
 - Description: There is continuous water leakage under the kitchen sink.
 - Location: Flat A402, Kitchen
 - Priority: MEDIUM
 - Status: OPEN
 - Category: Water Leakage
 - Resident: Rakesh Kumar (ID: 17)
 - Assigned Staff: None
 - Attachments: None
 - Feedback: None
 - Timeline: Initial complaint creation entry recorded with status OPEN and comment "Complaint created."
- Response contained "success": true

RESULT: SUCCESS



API ENDPOINT: POST /complaints/{complaint_id}/attachments

USER STORY: User Story 1 (As a Resident, I want to submit a complaint with its description, category, and photos, so that maintenance issues can be reported accurately and quickly.)

URL BEING TESTED: <http://127.0.0.1:5000/complaints/20/attachments>

DESCRIPTION: This API allows a resident to upload supporting documents or images related to a complaint. These attachments provide additional evidence to help the maintenance staff understand and resolve the reported issue. The API validates that a file is included in the request before processing the upload.

TEST CASE 1:

INPUTS:

- Input Parameter: Complaint ID-20

EXPECTED OUTPUT:

- HTTP Status Code: 400 Bad Request
- The upload request should be rejected because no file was provided.
- The API should return an appropriate validation error message.
- No attachment should be associated with the complaint.

ACTUAL OUTPUT:

- Curl**

```
curl -X 'POST' \
'http://127.0.0.1:5000/complaints/20/attachments' \
-H 'accept: application/json' \
-H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJmcWZhcjEwMlhdClBMTkNTU3HjMAMSwianRpTjo1NDQ0OTZmNzU0bzgxbS00PWE1Llg4ZWVtYnV1IiwiaWF0IjEwZSI6ImFjv2ZvcyIsInN1ViI6IjE3Iiwibm9udCkiOiJkaW5pdGVzIGoifQ==' \
-d ''
```

Request URL

```
http://127.0.0.1:5000/complaints/20/attachments
```

Server response

Code	Details
400	Error: BAD REQUEST Response body <pre>{ "success": false, "message": "No file provided." }</pre> Response headers <pre>access-control-allow-origin: http://127.0.0.1:5000 connection: close content-length: 61 content-type: application/json date: Sat, 01 Aug 2026 11:43:49 GMT server: Werkzeug/3.1.8 Python/3.13.1 vary: Origin</pre>

Responses

Code	Description
201	Uploaded
400	Bad request
403	Forbidden
404	Not found

USER STORY: Additional

DESCRIPTION: This API allows an **Administrator** to create a new complaint category in the Smart Society Management System. Complaint categories help classify maintenance issues (such as Electrical, Plumbing, Security, Gardening, etc.) so that complaints can be routed to the appropriate maintenance staff. Only authenticated administrators are authorized to access this endpoint.

INPUTS:

EXPECTED OUTPUT:

- HTTP Status Code: 201 Created
- A new complaint category should be created successfully.

- ACTUAL OUTPUT:**

- RESULT: SUCCESS**

API ENDPOINT : GET /admin/complaints

USER STORY: Additional

URL BEING TESTED: http://127.0.0.1:5000/admin/complaints?priority=Medium

DESCRIPTION: This API allows an **Administrator** to retrieve and monitor complaints submitted by residents. Administrators can filter complaints using query parameters such as priority, status, category, or pagination. The API returns complete complaint information, including resident details, category, assigned staff, attachments, feedback, and pagination details. This endpoint helps administrators monitor complaint resolution and assign work efficiently.

TEST CASE 1:

INPUTS:

- Input Parameter: Priority= Medium

EXPECTED OUTPUT:

- HTTP Status Code: 200 OK
- The API should retrieve complaints matching the specified priority.
- Each complaint should include:
 - Complaint ID
 - Complaint Reference Number
 - Title
 - Description
 - Priority
 - Status
 - Category Details
 - Resident Details
 - Assigned Staff Details (if available)
 - Attachments
 - Feedback
- Pagination details should also be returned.

ACTUAL OUTPUT:

- HTTP Status Code: 200 OK
- Complaint list retrieved successfully.
- The API returned complaint records containing:
 - Complaint ID
 - Complaint Reference Number
 - Complaint Title
 - Description
 - Location
 - Priority
 - Status
 - Category Details
 - Resident Details
 - Assigned Staff Details (where applicable)
 - Attachments
 - Feedback

- Pagination information returned:
 - **Page:** 1
 - **Records per Page:** 10
 - **Total Complaints:** 19
 - **Total Pages:** 2
 - **Has Next Page:** True
 - **Has Previous Page:** False
- Response contained "success": true.

RESULT: SUCCESS

The screenshot shows a web browser interface with the following details:

- Request URL:** `http://127.0.0.1:5000/admin/complaints?priority=Medium`
- Server response:**
 - Code:** 200
 - Response body:**

```
{
  "success": true,
  "complaints": [
    {
      "id": 10,
      "complaint_code": "CMP-2026-000009",
      "title": "Water leakage in kitchen",
      "description": "There is continuous water leakage under the kitchen sink.",
      "location": "Flat A401, Kitchen",
      "priority": "MEDIUM",
      "status": "CLOSED",
      "resident_id": 10,
      "assigned_staff_id": null,
      "category_id": 1,
      "created_at": "2026-07-31T15:12:04.519683",
      "updated_at": "2026-07-31T15:14:26.960126",
      "resolved_at": "2026-07-31T15:14:27.830816",
      "category": {
        "id": 1,
        "name": "Water Leakage",
        "description": "Water Leakage and pipe related issues",
        "is_active": true,
        "created_at": "2026-07-26T09:11:39.285241",
        "updated_at": "2026-07-26T09:11:39.285241"
      },
      "resident": {
        "id": 10,
        "name": "Resident 10",
        "email": "resident10@example.com",
        "phone": "9876543210"
      }
    }
  ]
}
```
 - Response headers:**

```
access-control-allow-origin: *
connection: close
content-length: 14766
content-type: application/json
date: Fri, 31 Jul 2026 18:54:18 GMT
server: Werkzeug/3.1.8 Python/3.13.1
```
- Responses:**

Code	Description
200	Success

API ENDPOINT : GET /admin/dashboard

USER STORY: **User Story 14** (As a Chairman, I want to view a dashboard showing pending, resolved, and overdue complaints, so that I can monitor the overall maintenance performance of the association.)

URL BEING TESTED: `http://127.0.0.1:5000/admin/dashboard`

DESCRIPTION: This API provides an overview of the Smart Society Management System for administrators. It returns key dashboard metrics such as the total number of complaints, complaint status distribution, priority distribution, total residents, total maintenance staff, average resident feedback rating, and a list of recent complaints. The dashboard enables administrators to monitor system performance and make informed management decisions.

TEST CASE 1:

INPUTS:

- Request method: GET

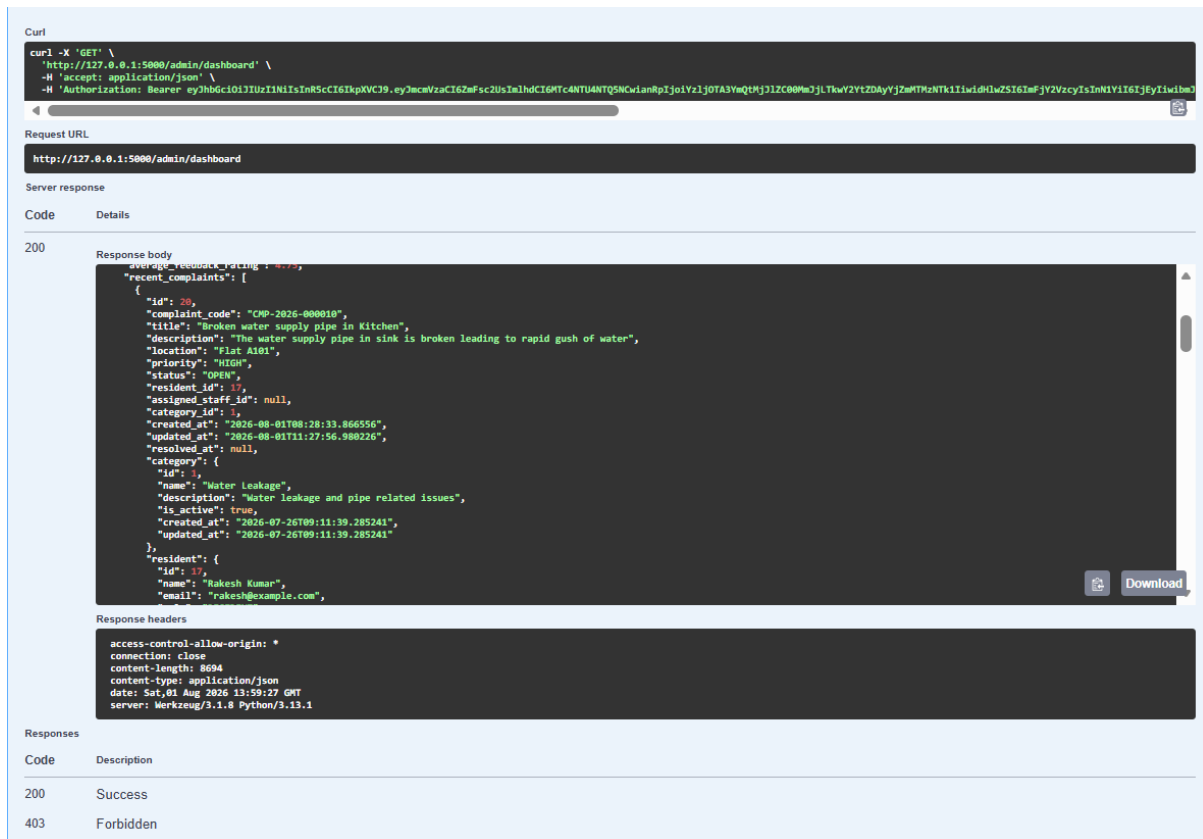
EXPECTED OUTPUT:

- HTTP Status Code: **200 OK**
- The administrator dashboard should be displayed successfully.
- The response should include:
 - Total complaints
 - Complaint status distribution
 - Complaint priority distribution
 - Total residents
 - Total maintenance staff
 - Average feedback rating
 - List of recent complaints with their details

ACTUAL OUTPUT:

- HTTP Status Code: **200 OK**
- Dashboard retrieved successfully.
- Dashboard statistics returned:
- Total Complaints: 20
- Complaint Status Distribution:
 - OPEN: 8
 - ASSIGNED: 4
 - IN_PROGRESS: 2
 - RESOLVED: 2
 - CLOSED: 4
 - REOPENED: 0
- Complaint Priority Distribution:
 - LOW: 3
 - MEDIUM: 6
 - HIGH: 9
 - URGENT: 2
- **Total Residents: 8**
- **Total Staff: 6**
- **Average Feedback Rating: 4.75**
- **Recent Complaints:** Details of the five most recent complaints, including complaint code, title, category, resident information, assigned staff (if any), attachments, and feedback.
- Response contained "success": true.

RESULT: SUCCESS



API ENDPOINT : POST /admin/staff

USER STORY: User Story 6 (As a Society Manager, I want to assign complaints to maintenance staff based on their expertise, so that issues are resolved by qualified personnel.)

URL BEING TESTED: http://127.0.0.1:5000/admin/staff

DESCRIPTION: This API allows an **Administrator** to create a new maintenance staff account in the Smart Society Management System. The administrator provides the staff member's personal information, login credentials, apartment details, and area of expertise (trade). Upon successful creation, the staff account is added to the system and can subsequently be assigned maintenance complaints based on the specified trade.

TEST CASE 1:

INPUTS:

```
{
  "name": "Raju Srivastava",
  "email": "staff2@smartsociety.com",
  "mobile_number": "9876543211",
  "flat_number": "A-101",
  "building": "Service Quarters",
}
```

```
"password": "Staff@123",  
"trade": "Plumber"  
}
```

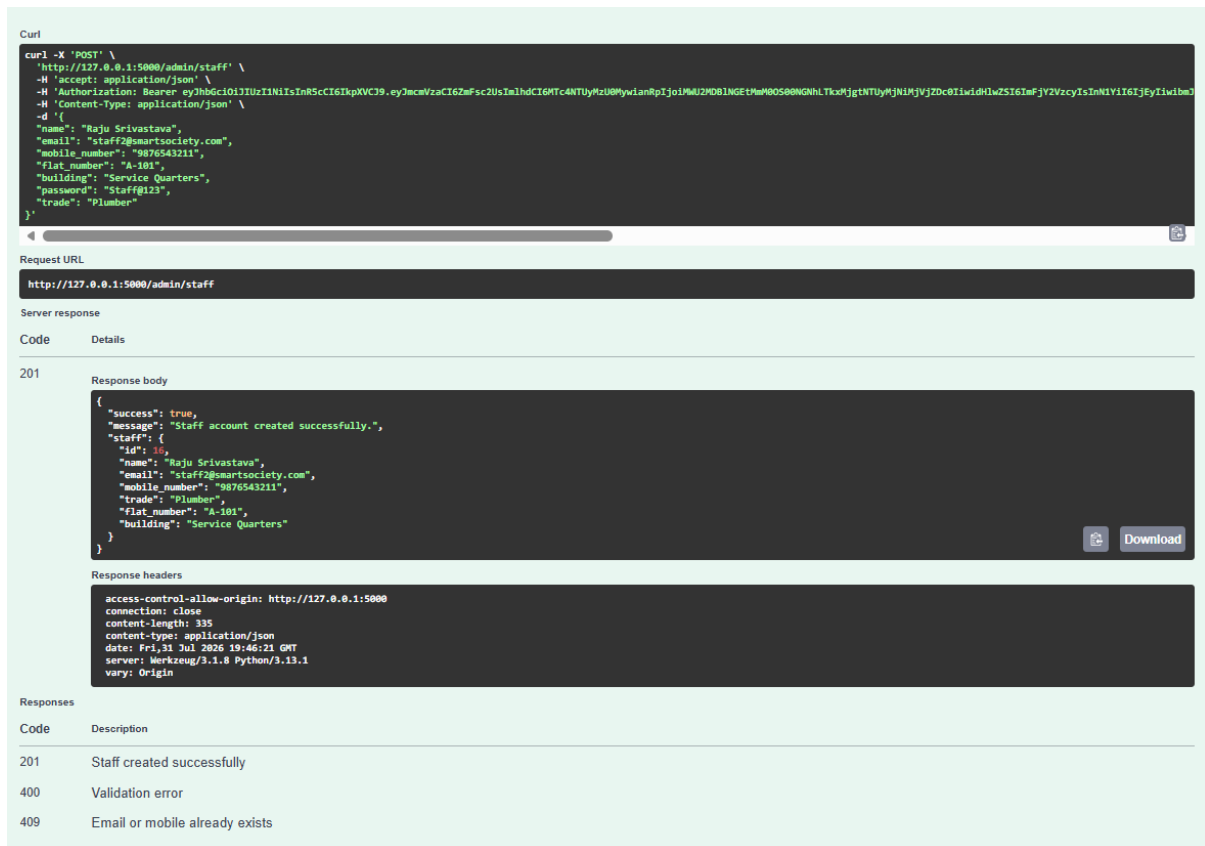
EXPECTED OUTPUT:

- HTTP Status Code: **201 Created**
- A new staff account should be created successfully.
- The response should return:
 - Staff ID
 - Staff Name
 - Email Address
 - Mobile Number
 - Trade (Specialization)
 - Flat Number
 - Building Name
- Success message: **"Staff account created successfully."**

ACTUAL OUTPUT:

- HTTP Status Code: **201 Created**
- Staff account created successfully.
- Staff details returned:
 - **Staff ID:** 16
 - **Name:** Raju Srivastava
 - **Email:** staff2@smartsociety.com
 - **Mobile Number:** 9876543211
 - **Trade:** Plumber
 - **Flat Number:** A-101
 - **Building:** Service Quarters
- Success message: **"Staff account created successfully."**

RESULT: SUCCESS



API ENDPOINT : PUT /admin/complaints/{complaint_id}/assign

USER STORY: User Story 6 (As a Society Manager, I want to assign complaints to maintenance staff based on their expertise, so that issues are resolved by qualified personnel.)

URL BEING TESTED: <http://127.0.0.1:5000/admin/complaints/20/assign>

DESCRIPTION: This API allows an **Administrator** to assign a maintenance staff member to a complaint. Once a valid staff member is assigned, the complaint status is updated to **ASSIGNED**, and the assigned staff member becomes responsible for resolving the issue. This API supports efficient complaint routing based on staff expertise and availability.

TEST CASE 1:

INPUTS:

Input Parameters:

Complaint ID: 20

Staff ID: 16

EXPECTED OUTPUT:

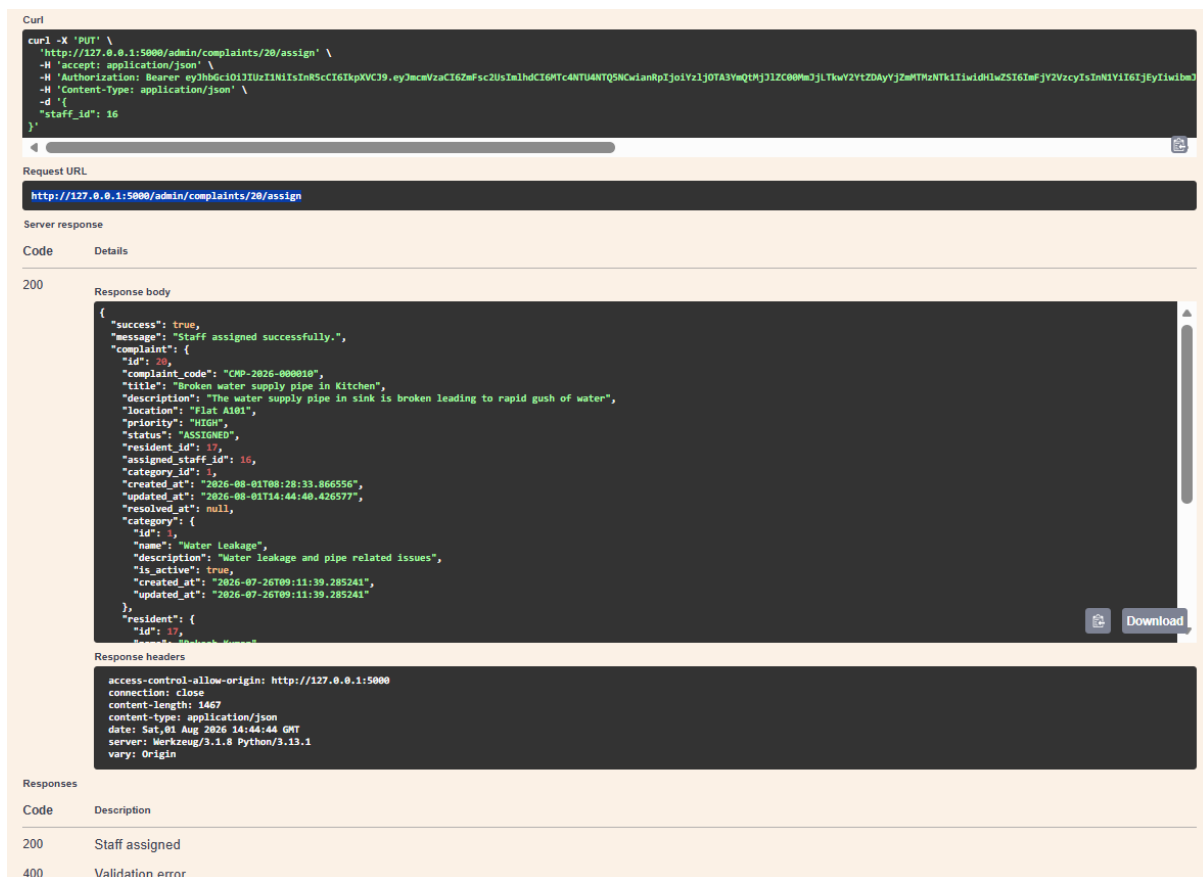
- HTTP Status Code: **200 OK**

- Complaint should be assigned successfully to the specified staff member.
- Complaint status should change from **OPEN** to **ASSIGNED**.
- The assigned staff details should be returned.
- The updated complaint information should be included in the response.
- Success message: "**Staff assigned successfully.**"

ACTUAL OUTPUT:

- HTTP Status Code: **200 OK**
- Staff assigned successfully.
- Complaint details updated:
 - **Complaint ID:** 20
 - **Complaint Code:** CMP-2026-000010
 - **Title:** Broken water supply pipe in Kitchen
 - **Status:** ASSIGNED
 - **Assigned Staff ID:** 16
- Assigned staff details returned:
 - **Staff ID:** 16
 - **Name:** Raju Srivastava
 - **Email:** staff2@smartsociety.com
 - **Role:** STAFF
 - **Flat Number:** A-101
 - **Building:** Service Quarters
- Success message: "**Staff assigned successfully.**"

RESULT: SUCCESS



API ENDPOINT : GET /complaints

USER STORY: User Story 7(As a Maintenance staff, I want to receive digital work orders with complaint details, priority, resident information and location so that I clearly understand the maintenance requirements.)

URL BEING TESTED:

http://127.0.0.1:5000/complaints?status=ASSIGNED&priority=HIGH&category_id=1

DESCRIPTION: This API allows an authenticated maintenance staff member to view complaints assigned to them. The API supports filtering complaints using query parameters such as complaint status, priority, and category. It returns complete complaint information, including resident details, category, assigned staff information, attachments, and feedback, enabling staff to efficiently manage their assigned maintenance tasks.

TEST CASE 1:

INPUTS:

Input Parameters:

status = ASSIGNED
priority = HIGH
category_id = 1

EXPECTED OUTPUT:

- HTTP Status Code: **200 OK**
- The API should return complaints matching the specified filters.
- Only complaints assigned to the authenticated staff member should be displayed.
- Each complaint should include:
 - Complaint ID
 - Complaint Reference Number
 - Title
 - Description
 - Location
 - Priority
 - Status
 - Category Details
 - Resident Details
 - Assigned Staff Details
 - Attachments
 - Feedback
- Pagination information should also be included.

ACTUAL OUTPUT:

- HTTP Status Code: 200 OK
- Complaint list retrieved successfully.
- One complaint matched the specified filters:
 - Complaint ID: 20
 - Complaint Code: CMP-2026-000010
 - Title: Broken water supply pipe in Kitchen
 - Priority: HIGH
 - Status: ASSIGNED
 - Category: Water Leakage
 - Resident: Rakesh Kumar (ID: 17)
 - Assigned Staff: Raju Srivastava (ID: 16)
 - Attachments: None
 - Feedback: None
- Pagination details:
 - Page: 1
 - Records Per Page: 10
 - Total Complaints: 1
 - Total Pages: 1
- Response contained "success": true

RESULT: SUCCESS

[illegible]

API ENDPOINT : GET /complaints/{complaint_id}

USER STORY: User Story 7(As a Maintenance staff, I want to receive digital work orders with complaint details, priority, resident information and location so that I clearly understand the maintenance requirements.)

URL BEING TESTED: <http://127.0.0.1:5000/complaints/20>

DESCRIPTION: This API allows an authenticated maintenance staff member to retrieve the complete details of a specific assigned complaint using its Complaint ID. The response includes complaint information, resident details, category, location, priority, assigned staff information, attachments, feedback, and the complaint timeline. It enables maintenance staff to understand the maintenance requirements before beginning work.

TEST CASE 1:

INPUTS:

Input Parameters:

Complaint ID = 20

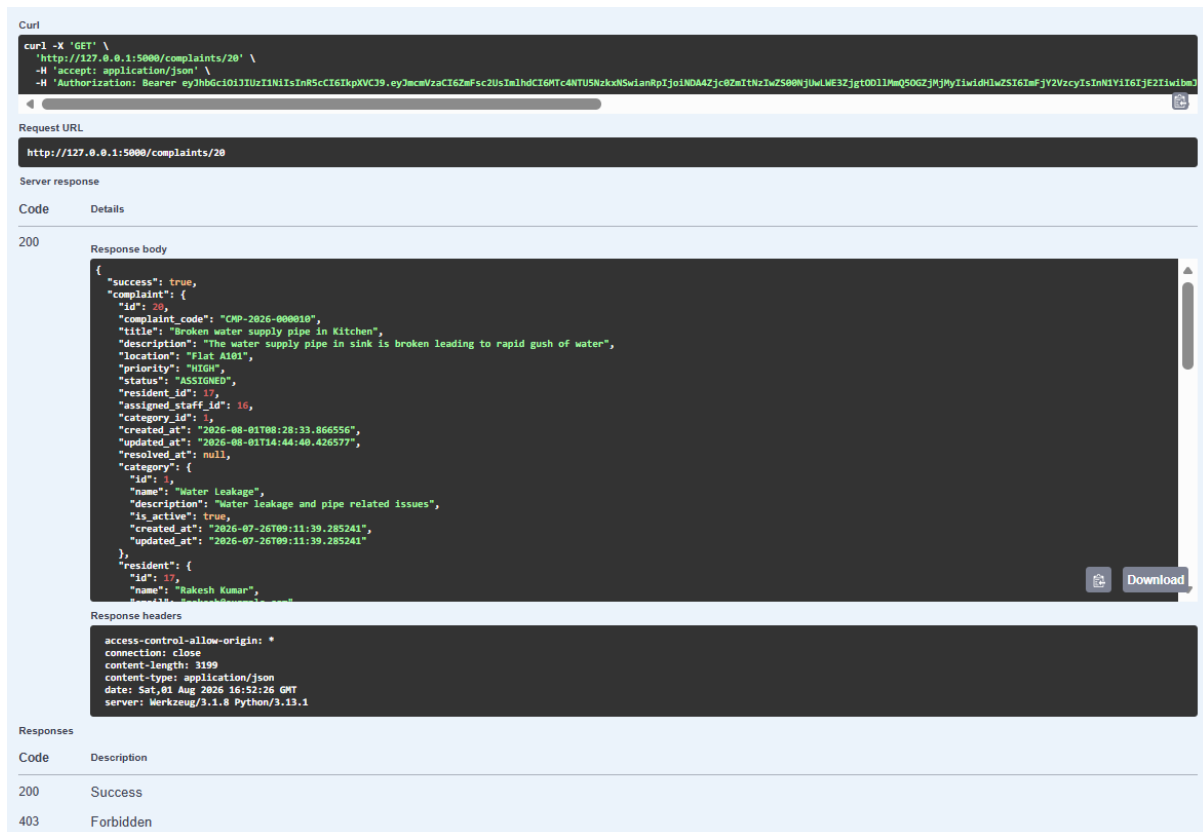
EXPECTED OUTPUT:

- HTTP Status Code: **200 OK**
- Complaint details should be retrieved successfully.
- The response should include:
 - Complaint ID
 - Complaint Reference Number
 - Title
 - Description
 - Location
 - Priority
 - Status
 - Category Details
 - Resident Details
 - Assigned Staff Details
 - Attachments
 - Feedback
 - Complaint Timeline

ACTUAL OUTPUT:

- HTTP Status Code: **200 OK**
- Complaint retrieved successfully.
- Complaint details returned:
 - Complaint ID: 20
 - Complaint Code: CMP-2026-000010
 - Title: Broken water supply pipe in Kitchen
 - Description: The water supply pipe in sink is broken leading to rapid gush of water
 - Location: Flat A101
 - Priority: HIGH
 - Status: ASSIGNED
 - Category: Water Leakage
 - Resident: Rakesh Kumar (ID: 17)
 - Assigned Staff: Raju Srivastava (ID: 16)
 - Attachments: None
 - Feedback: None
 - Timeline:
 - Complaint created by Resident.
 - Complaint updated by Resident.
 - Complaint assigned to Staff by Admin.
- Response contained "success": true

RESULT: SUCCESS



API ENDPOINT : PUT /staff/complaints/{complaint id}/status

USER STORY: User Story 8(As a Maintenance staff, I want to update the status of assigned tasks, so that management and residents can track progress effectively.)

URL BEING TESTED: <http://127.0.0.1:5000/staff/complaints/20/status>

DESCRIPTION: This API allows a maintenance staff member to update the status of an assigned complaint and provide a progress comment. The system enforces the complaint workflow by allowing only valid status transitions (e.g., **ASSIGNED** → **IN_PROGRESS** → **RESOLVED**). Invalid status transitions are rejected with an appropriate error message.

TEST CASE 1:

INPUTS:

Input Parameters:

Complaint ID = 20

 $\{$

```
"status": "RESOLVED",
```

```
"comment": "The broken pipe was fixed and checked for leakage. No leakage found after 30 minutes."
```

}

EXPECTED OUTPUT:

- HTTP Status Code: **400 Bad Request**
- The complaint status should **not** be updated because the complaint is currently in the **ASSIGNED** state.
- The API should enforce the workflow by requiring the complaint to be moved to **IN_PROGRESS** before it can be marked **RESOLVED**.
- An appropriate validation message should be returned.

ACTUAL OUTPUT:

- HTTP Status Code: **400 Bad Request**
- Complaint status update rejected.
- Error message returned: "Complaint must be IN_PROGRESS before marking as RESOLVED."
- Complaint status remained unchanged

RESULT: SUCCESS

Curl

```
curl -X 'PUT' \
  'https://127.0.0.1:5000/staff/complaints/20/status' \
  -H 'accept: application/json' \
  -H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJmcWVzaCI6ZmFsc2UsIm1hdCI6MTc4NTU5NzIxNzIwZS00MjUwLWV3ZjgtODI1MmQ5OGZjMjMyIiwiaWF0IjE2Iiwibm9iIjoiIn0=' \
  -H 'Content-Type: application/json' \
  -d '{
    "status": "RESOLVED",
    "comment": "The broken pipe was fixed and checked for leakage, No leakage found after 30 minutes."
  }'
```

Request URL

<http://127.0.0.1:5000/staff/complaints/20/status>

Server response

Code Details

400 Error: BAD REQUEST

Response body

```
{
  "success": false,
  "message": "Complaint must be IN_PROGRESS before marking as RESOLVED."
}
```

Response headers

```
access-control-allow-origin: http://127.0.0.1:5000
connection: close
content-length: 101
content-type: application/json
date: Sat, 01 Aug 2026 17:16:09 GMT
server: Werkzeug/3.1.8 Python/3.13.1
vary: Origin
```

Responses

Code	Description
200	Status updated
400	Validation error
403	Forbidden
404	Not found

TEST CASE 2:

INPUTS:

Input Parameters:

Complaint ID = 20

{

"status": "IN_PROGRESS",

```
"comment": "The broken pipe was fixed and checked for leakage. No leakage found after 30 minutes."
}
```

EXPECTED OUTPUT:

- HTTP Status Code: **200 OK**
- Complaint status should be updated successfully.
- Complaint status should change from **ASSIGNED** to **IN_PROGRESS**.
- The progress comment should be recorded in the complaint timeline.
- Updated complaint details should be returned.
- Success message: **"Complaint status updated successfully."**

ACTUAL OUTPUT:

- HTTP Status Code: **200 OK**
- Complaint status updated successfully.
- Complaint details returned:
 - **Complaint ID:** 20
 - **Complaint Code:** CMP-2026-000010
 - **Title:** Broken water supply pipe in Kitchen
 - **Status:** IN_PROGRESS
 - **Priority:** HIGH
 - **Category:** Water Leakage
 - **Resident:** Rakesh Kumar (ID: 17)
 - **Assigned Staff:** Raju Srivastava (ID: 16)
 - **Updated At:** 2026-08-01T17:26:07.819227
- Success message: **"Complaint status updated successfully."**

RESULT: SUCCESS

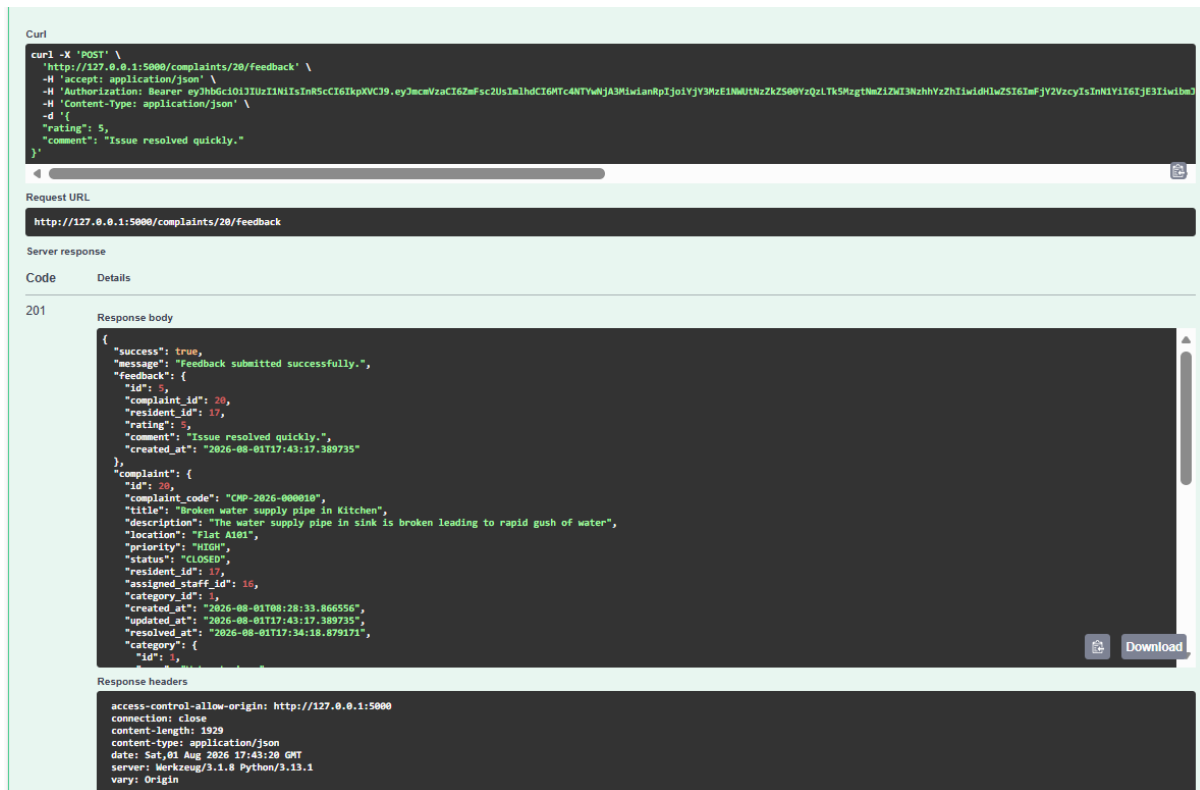
EXPECTED OUTPUT:

- HTTP Status Code: **201 Created**
- Feedback should be submitted successfully.
- The rating and comment should be stored against the complaint.
- The complaint should remain in the **CLOSED** state.
- The response should return:
 - Feedback ID
 - Complaint ID
 - Resident ID
 - Rating
 - Comment
 - Submission Timestamp
- Success message: "**Feedback submitted successfully.**"

ACTUAL OUTPUT:

- HTTP Status Code: 201 Created
- Feedback submitted successfully.
- Feedback details returned:
 - Feedback ID: 5
 - Complaint ID: 20
 - Resident ID: 17
 - Rating: 5
 - Comment: "Issue resolved quickly."
 - Created At: 2026-08-01T17:43:17.389735
- Complaint status remained **CLOSED**.
- Complaint details and submitted feedback were returned successfully

RESULT: SUCCESS



API ENDPOINT : GET /notifications

USER STORY: User Story 5(As a Resident, I want to receive notifications whenever my complaint status changes, so that I stay informed without contacting management.)

URL BEING TESTED: <http://127.0.0.1:5000/notification>

DESCRIPTION: This API allows an authenticated resident to retrieve all notifications associated with their account. The notifications include important updates such as staff assignments, complaint status changes, and feedback confirmations. Each notification contains its title, message, notification type, delivery channel, read/unread status, and creation timestamp. The API also returns the total number of unread notifications and pagination details.

TEST CASE 1:

INPUTS:

- Request method = GET

EXPECTED OUTPUT:

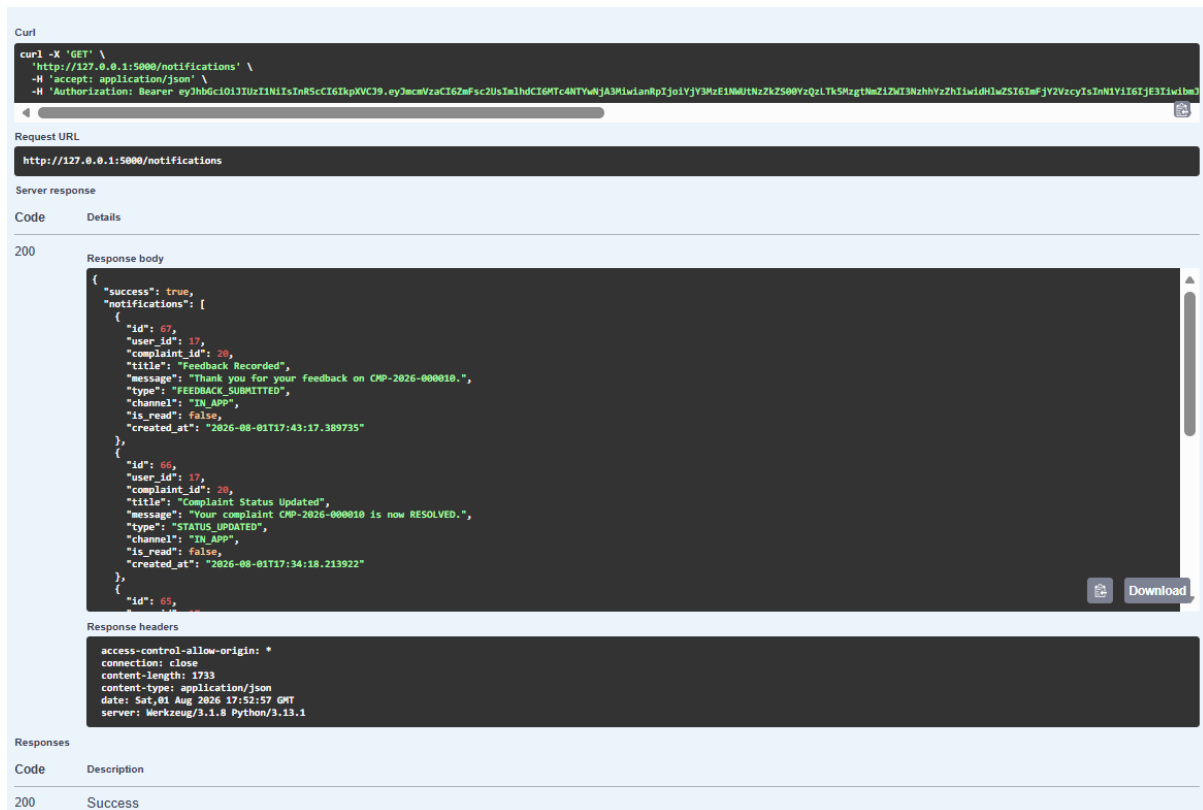
- HTTP Status Code: 200 OK
- All notifications belonging to the authenticated resident should be retrieved.
- Each notification should include:
 - Notification ID
 - Complaint ID
 - Title

- Message
- Notification Type
- Delivery Channel
- Read/Unread Status
- Creation Timestamp
- The response should also include:
 - Total unread notifications
 - Pagination details

ACTUAL OUTPUT:

- HTTP Status Code: **200 OK**
- Notifications retrieved successfully.
- Four notifications were returned:
 1. **Feedback Recorded** – Thank you for your feedback on complaint **CMP-2026-000010**.
 2. **Complaint Status Updated** – Complaint status changed to **RESOLVED**.
 3. **Complaint Status Updated** – Complaint status changed to **IN_PROGRESS**.
 4. **Staff Assigned** – Staff member **Raju Srivastava** assigned to complaint **CMP-2026-000010**.
- **Unread Notifications:** 4
- **Pagination details:**
 - Page: 1
 - Per Page: 20
 - Total Notifications: 4
 - Total Pages: 1
- Response contained "success": true

RESULT: SUCCESS



API ENDPOINT : PUT /notifications/{notification_id}/read

USER STORY: User Story 5(As a Resident, I want to receive notifications whenever my complaint status changes, so that I stay informed without contacting management.)

URL BEING TESTED: <http://127.0.0.1:5000/notifications/61/read>

DESCRIPTION: This API allows an authenticated resident to mark a specific notification as **read**. Once updated, the notification's read status changes from **unread** to **read**, enabling residents to manage and track which notifications they have already viewed.

TEST CASE 1:

INPUTS:

Notification ID = 61

EXPECTED OUTPUT:

- HTTP Status Code: **200 OK**
- The specified notification should be marked as **read**.
- The API should return the updated notification details.
- The `is_read` field should be set to **true**.
- Success message: **"Notification marked as read."**

ACTUAL OUTPUT:

- HTTP Status Code: **200 OK**
- Notification marked as read successfully.
- Updated notification details returned:
 - Notification ID: 61
 - User ID: 17
 - Complaint ID: 20
 - Title: Staff Assigned
 - Message: Staff member Raju Srivastava has been assigned to your complaint CMP-2026-000010.
 - Type: STAFF_ASSIGNED
 - Channel: IN_APP
 - Read Status: True
 - Created At: 2026-08-01T14:44:40.426577
- Success message: **"Notification marked as read."**

RESULT: SUCCESS

Curl

```
curl -X 'PUT' \
'http://127.0.0.1:5000/notifications/61/read' \
-H 'accept: application/json' \
-H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJncmVzaCI6ZmFsc2UsImhhbmQiOjEwMTUzZkZ5MDYzLlR5cywgImN1YXI6IjE3IiwiaWF0IjoiMjAyNi00OTI1LTQ0LTQ0LTIyOjA0LjQ2NTUyInQ'`
```

Request URL

<http://127.0.0.1:5000/notifications/61/read>

Server response

Code	Details
200	Response body <pre>{ "success": true, "message": "Notification marked as read.", "notification": { "id": 61, "user_id": 17, "complaint_id": 20, "title": "Staff Assigned", "message": "Staff member Raju Srivastava has been assigned to your complaint CMP-2026-000010.", "type": "STAFF_ASSIGNED", "channel": "IN_APP", "is_read": true, "created_at": "2026-08-01T14:44:40.426577" } }</pre> Response headers <pre>access-control-allow-origin: http://127.0.0.1:5000 connection: close content-length: 447 content-type: application/json date: Sat, 01 Aug 2026 18:30:57 GMT server: Werkzeug/3.1.8 Python/3.13.1 vary: Origin</pre>
404	Not found

Responses

Code	Description
200	Marked as read
404	Not found

API ENDPOINT : PUT /notifications/read-all

USER STORY: User Story 5(As a Resident, I want to receive notifications whenever my complaint status changes, so that I stay informed without contacting management.)

URL BEING TESTED: <http://127.0.0.1:5000/notifications/read-all>

DESCRIPTION: This API allows an authenticated resident to mark all unread notifications associated with their account as **read**. It updates the read status of every unread notification in a single operation, enabling residents to quickly clear their notification list after reviewing all updates.

TEST CASE 1:

INPUTS:

Notification ID = 61

EXPECTED OUTPUT:

- HTTP Status Code: **200 OK**
- All unread notifications should be marked as **read**.
- The API should return a confirmation message indicating that the operation was successful.

ACTUAL OUTPUT:

- HTTP Status Code: **200 OK**
- All unread notifications were successfully marked as **read**.
- Success message returned: "**All notifications marked as read.**"

RESULT: SUCCESS

The screenshot displays a REST client interface with the following details:

- Request URL:** `http://127.0.0.1:5000/notifications/read-all`
- Server response:**
 - Code:** 200
 - Response body:**

```
{
  "success": true,
  "message": "All notifications marked as read."
}
```
 - Response headers:**

```
access-control-allow-origin: http://127.0.0.1:5000
connection: close
content-length: 76
content-type: application/json
date: Sat, 01 Aug 2020 19:40:24 GMT
server: Werkzeug/3.1.8 Python/3.13.1
vary: Origin
```
- Responses table:**

Code	Description
200	All marked as read

SNAPSHOTS OF PYTESTS

This screenshot shows the initial setup in VS Code. The Explorer panel on the left displays the project structure, including files like `report.html` and `requirements.txt`. The Terminal panel at the bottom shows the execution of `pip install pytest-html`, which successfully installs the package. Below this, the command `pytest -v --html=report.html --self-contained-html` is run, starting a test session. The output shows the collection of 54 items and the beginning of test execution for `tests/test_admin.py`.

```
(venv) PS D:\SOFTWARE ENGINEERING PROJECT MAY 26\codes\WAY2026-Team-095\backend> pip install pytest-html
Using cached pytest_html-4.2.0-py3-none-any.whl (23 kB)
Using cached pytest_metadata-3.1.1-py3-none-any.whl (11 kB)
Installing collected packages: pytest-metadata, pytest-html
Successfully installed pytest-html-4.2.0 pytest-metadata-3.1.1

[notice] A new release of pip is available: 24.3.1 -> 26.2
[notice] To update, run: python.exe -m pip install --upgrade pip
(venv) PS D:\SOFTWARE ENGINEERING PROJECT MAY 26\codes\WAY2026-Team-095\backend> pytest -v --html=report.html --self-contained-html
===== test session starts =====
platform win32 -- Python 3.13.1, pytest-9.1.1, pluggy-1.6.0 -- D:\SOFTWARE ENGINEERING PROJECT MAY 26\codes\WAY2026-Team-095\backend\venv\Scripts\python.exe
cachedir: .pytest_cache
metadata: {'Python': '3.13.1', 'Platform': 'Windows-11-10.0.26200-SP0', 'Packages': {'pytest': '9.1.1', 'pluggy': '1.6.0'}, 'Plugins': {'flask': '1.3.0', 'html': '4.2.0', 'metadata': '3.1.1'}}
rootdir: D:\SOFTWARE ENGINEERING PROJECT MAY 26\codes\WAY2026-Team-095\backend
plugins: flask-1.3.0, html-4.2.0, metadata-3.1.1
collected 54 items

tests/test_admin.py::test_admin_dashboard PASSED [ 1%]
tests/test_admin.py::test_get_staff PASSED [ 3%]
tests/test_admin.py::test_create_category PASSED [ 5%]
tests/test_admin.py::test_create_staff PASSED [ 7%]
tests/test_admin.py::test_assign_staff PASSED [ 9%]
tests/test_auth.py::test_login_success PASSED [ 11%]
tests/test_auth.py::test_login_invalid_password PASSED [ 13%]
tests/test_auth.py::test_login_invalid_email PASSED [ 14%]
tests/test_auth.py::test_profile_success PASSED [ 16%]
tests/test_auth.py::test_profile_without_token PASSED [ 18%]
tests/test_auth.py::test_profile_invalid_token PASSED [ 20%]
tests/test_categories.py::test_get_categories_resident PASSED [ 22%]
tests/test_categories.py::test_get_categories_admin PASSED [ 24%]
tests/test_categories.py::test_get_categories_staff PASSED [ 26%]
tests/test_categories.py::test_get_categories_without_token PASSED [ 27%]
tests/test_categories.py::test_get_categories_invalid_token PASSED [ 29%]
tests/test_complaints.py::test_create_complaint_success PASSED [ 31%]
tests/test_complaints.py::test_create_complaint_invalid_category PASSED [ 33%]
tests/test_complaints.py::test_create_complaint_staff_forbidden PASSED [ 35%]
tests/test_complaints.py::test_create_complaint_admin_forbidden PASSED [ 37%]
tests/test_complaints.py::test_create_complaint_without_token PASSED [ 38%]
tests/test_complaints.py::test_create_complaint_invalid_token PASSED [ 40%]
```

This screenshot shows the continuation of the test session from the previous snapshot. The terminal displays the results of the remaining tests, all of which passed. The session concludes with a 'warnings summary' section, indicating that there were no warnings. The final output shows the completion of the test session for `tests/test_admin.py`.

```
tests/test_complaints.py::test_create_complaint_invalid_category PASSED [ 33%]
tests/test_complaints.py::test_create_complaint_staff_forbidden PASSED [ 35%]
tests/test_complaints.py::test_create_complaint_admin_forbidden PASSED [ 37%]
tests/test_complaints.py::test_create_complaint_without_token PASSED [ 38%]
tests/test_complaints.py::test_list_complaints_resident PASSED [ 40%]
tests/test_complaints.py::test_list_complaints_staff PASSED [ 42%]
tests/test_complaints.py::test_list_complaints_admin PASSED [ 44%]
tests/test_complaints.py::test_list_complaints_without_token PASSED [ 46%]
tests/test_complaints.py::test_list_complaints_invalid_token PASSED [ 48%]
tests/test_complaints.py::test_get_complaint_success PASSED [ 51%]
tests/test_complaints.py::test_get_complaint_invalid_id PASSED [ 53%]
tests/test_complaints.py::test_get_complaint_without_token PASSED [ 55%]
tests/test_complaints.py::test_get_complaint_invalid_token PASSED [ 57%]
tests/test_complaints.py::test_update_complaint_success PASSED [ 59%]
tests/test_complaints.py::test_update_complaint_invalid_id PASSED [ 61%]
tests/test_complaints.py::test_update_complaint_without_token PASSED [ 62%]
tests/test_complaints.py::test_update_complaint_invalid_token PASSED [ 64%]
tests/test_complaints.py::test_get_timeline_success PASSED [ 66%]
tests/test_complaints.py::test_get_timeline_invalid_id PASSED [ 68%]
tests/test_complaints.py::test_get_timeline_without_token PASSED [ 70%]
tests/test_complaints.py::test_get_timeline_invalid_token PASSED [ 72%]
tests/test_complaints.py::test_feedback_open_complaint PASSED [ 74%]
tests/test_complaints.py::test_feedback_without_token PASSED [ 75%]
tests/test_complaints.py::test_feedback_invalid_token PASSED [ 77%]
tests/test_complaints.py::test_upload_attachment_without_file PASSED [ 79%]
tests/test_complaints.py::test_upload_attachment_without_token PASSED [ 81%]
tests/test_complaints.py::test_upload_attachment_invalid_token PASSED [ 83%]
tests/test_notifications.py::test_get_notifications PASSED [ 85%]
tests/test_notifications.py::test_mark_notification_read PASSED [ 87%]
tests/test_notifications.py::test_mark_all_notifications_read PASSED [ 88%]
tests/test_notifications.py::test_notifications_without_token PASSED [ 90%]
tests/test_notifications.py::test_notifications_invalid_token PASSED [ 92%]
tests/test_staff.py::test_staff_view_complaint PASSED [ 94%]
tests/test_staff.py::test_update_status_in_progress PASSED [ 96%]
tests/test_staff.py::test_update_status_resolved PASSED [ 98%]
tests/test_staff.py::test_invalid_status_transition PASSED [ 100%]

===== warnings summary =====
tests/test_admin.py::test_admin_dashboard
```

pytest_report.htmlreport.html

Ask Gemini

Verify it's youNew Chrome available

report.html

Report generated on 02-Aug-2026 at 11:37:14 by [pytest-html](#) v4.2.0

Environment

Python	3.13.1
Platform	Windows-11-10.0.26200-SP0
Packages	• pytest: 8.1.1 • pluggy: 1.6.0
Plugins	• flask: 1.3.0 • html: 4.2.0 • metadata: 3.1.1

Summary

54 tests took 00:01:00.

(Un)check the boxes to filter the results

0 Failed

54 Passed

0 Skipped

0 Expected failures

0 Unexpected passes

0 Errors

0 Reruns

0 Retired

Show all details / Hide all details

Result	Test	Duration	Links
Passed	tests/test_admin.py::test_admin_dashboard	00:00:09	
Passed	tests/test_admin.py::test_get_staff	00:00:04	
Passed	tests/test_admin.py::test_create_category	00:00:01	
Passed	tests/test_admin.py::test_create_staff	00:00:02	
Passed	tests/test_admin.py::test_assign_staff	00:00:05	
Passed	tests/test_auth.py::test_login_success	271 ms	
Passed	tests/test_auth.py::test_login_invalid_password	337 ms	
Passed	tests/test_auth.py::test_login_invalid_email	207 ms	
Passed	tests/test_auth.py::test_profile_success	202 ms	
Passed	tests/test_auth.py::test_profile_without_token	3 ms	
Passed	tests/test_auth.py::test_profile_invalid_token	3 ms	
Passed	tests/test_categories.py::test_get_categories_resident	289 ms	
Passed	tests/test_categories.py::test_get_categories_admin	202 ms	
Passed	tests/test_categories.py::test_get_categories_staff	510 ms	

World Cup cov...
FIFA World Cup...

Search

ENG
IN

14:00
02-08-2026

pytest_report.htmlreport.html

Ask Gemini

Verify it's youNew Chrome available

← → ↺ 📄 D:\SOFTWARE\20ENGINEERING\20PROJECT\20MAY\2026\codes\MAY2026-Team-095\backend\report.html?sort=result ☆ 📌

Passed	tests/test_categories.py::test_get_categories_admin	202 ms	
Passed	tests/test_categories.py::test_get_categories_staff	510 ms	
Passed	tests/test_categories.py::test_get_categories_without_token	1 ms	
Passed	tests/test_categories.py::test_get_categories_invalid_token	1 ms	
Passed	tests/test_complaints.py::test_create_complaint_success	00:00:02	
Passed	tests/test_complaints.py::test_create_complaint_invalid_category	176 ms	
Passed	tests/test_complaints.py::test_create_complaint_staff_forbidden	3 ms	
Passed	tests/test_complaints.py::test_create_complaint_admin_forbidden	2 ms	
Passed	tests/test_complaints.py::test_create_complaint_without_token	1 ms	
Passed	tests/test_complaints.py::test_create_complaint_invalid_token	1 ms	
Passed	tests/test_complaints.py::test_list_complaints_resident	00:00:04	
Passed	tests/test_complaints.py::test_list_complaints_staff	00:00:05	
Passed	tests/test_complaints.py::test_list_complaints_admin	00:00:05	
Passed	tests/test_complaints.py::test_list_complaints_without_token	1 ms	
Passed	tests/test_complaints.py::test_list_complaints_invalid_token	2 ms	
Passed	tests/test_complaints.py::test_get_complaint_success	00:00:01	
Passed	tests/test_complaints.py::test_get_complaint_invalid_id	376 ms	
Passed	tests/test_complaints.py::test_get_complaint_without_token	2 ms	
Passed	tests/test_complaints.py::test_get_complaint_invalid_token	2 ms	
Passed	tests/test_complaints.py::test_update_complaint_success	00:00:04	
Passed	tests/test_complaints.py::test_update_complaint_invalid_id	196 ms	
Passed	tests/test_complaints.py::test_update_complaint_without_token	2 ms	
Passed	tests/test_complaints.py::test_update_complaint_invalid_token	2 ms	
Passed	tests/test_complaints.py::test_get_timeline_success	585 ms	
Passed	tests/test_complaints.py::test_get_timeline_invalid_id	425 ms	
Passed	tests/test_complaints.py::test_get_timeline_without_token	1 ms	
Passed	tests/test_complaints.py::test_get_timeline_invalid_token	1 ms	
Passed	tests/test_complaints.py::test_feedback_open_complaint	199 ms	
Passed	tests/test_complaints.py::test_feedback_without_token	2 ms	
Passed	tests/test_complaints.py::test_feedback_invalid_token	2 ms	

World Cup cov...
FIFA World Cup...

Search

ENG
IN

14:00
02-08-2026

Passed	tests/test_complaints.py::test_get_timeline_without_token	1 ms	
Passed	tests/test_complaints.py::test_get_timeline_invalid_token	1 ms	
Passed	tests/test_complaints.py::test_feedback_open_complaint	199 ms	
Passed	tests/test_complaints.py::test_feedback_without_token	2 ms	
Passed	tests/test_complaints.py::test_feedback_invalid_token	2 ms	
Passed	tests/test_complaints.py::test_upload_attachment_without_file	397 ms	
Passed	tests/test_complaints.py::test_upload_attachment_without_token	2 ms	
Passed	tests/test_complaints.py::test_upload_attachment_invalid_token	2 ms	
Passed	tests/test_notifications.py::test_get_notifications	599 ms	
Passed	tests/test_notifications.py::test_mark_notification_read	00:00:02	
Passed	tests/test_notifications.py::test_mark_all_notifications_read	372 ms	
Passed	tests/test_notifications.py::test_notifications_without_token	3 ms	
Passed	tests/test_notifications.py::test_notifications_invalid_token	4 ms	
Passed	tests/test_staff.py::test_staff_view_complaint	00:00:06	
Passed	tests/test_staff.py::test_update_status_in_progress	00:00:02	
Passed	tests/test_staff.py::test_update_status_resolved	00:00:02	
Passed	tests/test_staff.py::test_invalid_status_transition	674 ms	

USER FEEDBACK

Based on user interactions and feedback received after the development of the application, the following suggestions were identified for future enhancements

1. Resident Approval Workflow

After a new resident registers, the Administrator should receive a notification requesting approval or rejection of the registration. A resident account should remain inactive until it has been approved by the Administrator. Only approved residents should be permitted to log in and raise complaints.

2. Category Management

The Administrator should have the ability to delete complaint categories that are no longer required or relevant. This would help maintain an up-to-date and organized list of complaint categories.

3. Category-Based Staff Assignment

At present, when assigning a complaint, the Administrator is presented with the complete list of maintenance staff. Instead, the system should automatically filter and display only those staff members whose expertise matches the selected complaint category (e.g., plumbers for plumbing issues and electricians for electrical issues). This would simplify the assignment process and ensure that complaints are allocated to the most appropriate maintenance personnel.

4. Complaint Submission Restricted to the Registered Flat

Currently, a resident is able to create or update a complaint for any flat within the society by manually entering a different flat number. This is not an appropriate practice. The system should automatically associate complaints with the resident's registered flat and building details, ensuring that residents can raise and manage complaints only for the flat to which they are registered. This enhancement would improve data integrity, security, and accountability within the complaint management process.

PLAN FOR NEXT SPRINT

During the second sprint, our primary focus will be on implementing the remaining seven user stories from Milestone 1. Furthermore, we will refine and enhance the existing APIs by addressing the feedback received from users, ensuring improved functionality, security, and overall system usability.